

# Private Set Union avec Chiffrement Homomorphe de Classe $\mathcal{CL}$

Intégration de BICYCL dans volePSI – Comparaison et Évaluation

Yasser NAMATY

Laboratoire Jean Kuntzmann

Encadrants : Aude Maignan / Jean-Guillaume Dumas

2026

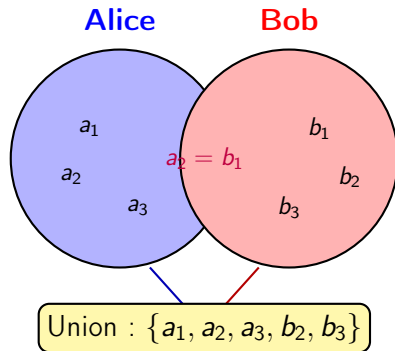
# Plan de la présentation

- 1 Contexte et problématique
- 2 Implémentation
- 3 Résultats expérimentaux
- 4 Analyse et perspectives

# Private Set Union (PSU)

## Problème

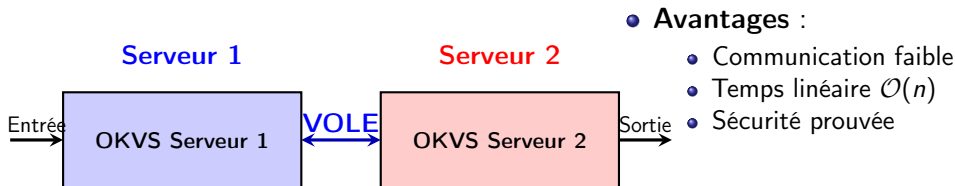
Deux parties, chacune détenant un ensemble d'éléments privés, souhaitent calculer l'union de leurs ensembles sans révéler d'informations supplémentaires.



- Applications :
  - Données médicales
  - Détection de menaces
  - Publicité ciblée
- **Objectif** : Calculer l'union sans révéler les éléments

## FPSU (Fast Private Set Union)

- Basé sur **volePSI** (VOLE-based PSI)
- Utilise des corrélations VOLE (Vector Oblivious Linear Evaluation)
- Encodage des ensembles via **OKVS** (Paxos)



# Chiffrement Homomorphe de Classe $\mathcal{CL}$

## Définition

- Groupe de classes d'idéaux de corps quadratiques imaginaires
- Schéma  $\mathcal{CL}$ -**HSMqk** (Castagnos-Laguillaumie)
- Chiffrement **additif** homomorphe

## Propriétés mathématiques

$$\text{Cl}(\Delta_K) = \frac{\text{Idéaux fractionnaires}}{\text{Idéaux principaux}}$$

$$\mathcal{E}(m_1 + m_2) = \mathcal{E}(m_1) \oplus \mathcal{E}(m_2)$$

## Caractéristiques

- Sécurité 128 bits :  $q = 256$  bits
- Chiffrés : 4663 bits
- Add homomorphe : 11 ms
- Clés compactes

## BICYCL

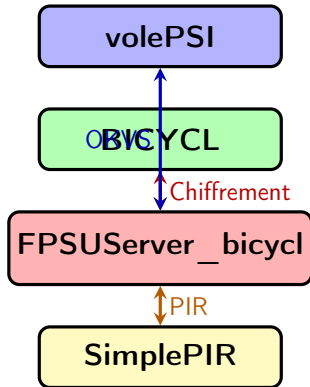
- Bibliothèque C++ du LIRMM
- Implémente  $\mathcal{CL}$ -HSMqk
- Interface moderne (C++20)

## Caractéristiques

- Opérations sur formes quadratiques (NUCOMP)
- Compatible avec volePSI
- Paramètres flexibles

## Pourquoi BICYCL ?

- Chiffrés plus compacts que Paillier
- Chiffrement plus rapide
- Meilleure intégration C++



- OKVS (Paxos)
- VOLE
- OT
- $\mathcal{CL}$ -HSMqk
- Requêtes PIR

## Définition

```
constexpr size_t kMaxValueBytes = 4096;  
constexpr size_t kMaxValueBlocks = kMaxValueBytes / sizeof(block);  
using ValueType = std::array<block, kMaxValueBlocks>;
```

## Opérateurs requis

- `operator^`
- `operator^=`
- `operator&`

## Sérialisation

- Chiffré  $\rightarrow$  ValueType
- ValueType  $\rightarrow$  Chiffré
- Taille fixe : 4096 octets

## Encodage compact

- Chaque QFI encode : (a, b, c)
- Marqueur de signe + longueur
- Taille totale : variable

```
std::vector<unsigned char> encode_mpz(const BICYCL::Mpz& value) {  
    std::vector<unsigned char> abs_bytes = value;  
    std::vector<unsigned char> out;  
    out.push_back(marker);  
    append_u32(out, abs_bytes.size());  
    out.insert(out.end(), abs_bytes.begin(), abs_bytes.end());  
    return out;  
}
```

## Format

|         |            |            |
|---------|------------|------------|
| Octet 0 | Octets 1-4 | Octets 5-N |
|---------|------------|------------|

## Modifications clés

- 1 Définition de `ValueType` adaptée
- 2 Sérialisation BICYCL  $\leftrightarrow$  OKVS
- 3 Reconstruction homomorphe des partages

## Flux du protocole

- 1 Chiffrement des partages :  $\mathcal{E}(s_1), \mathcal{E}(s_2)$  avec  $s_1 + s_2 = 0$
- 2 Encodage OKVS avec les chiffrés
- 3 Décodage et addition homomorphe
- 4 Vérification :  $\mathcal{D}(\mathcal{E}(s_1) + \mathcal{E}(s_2)) = 0$

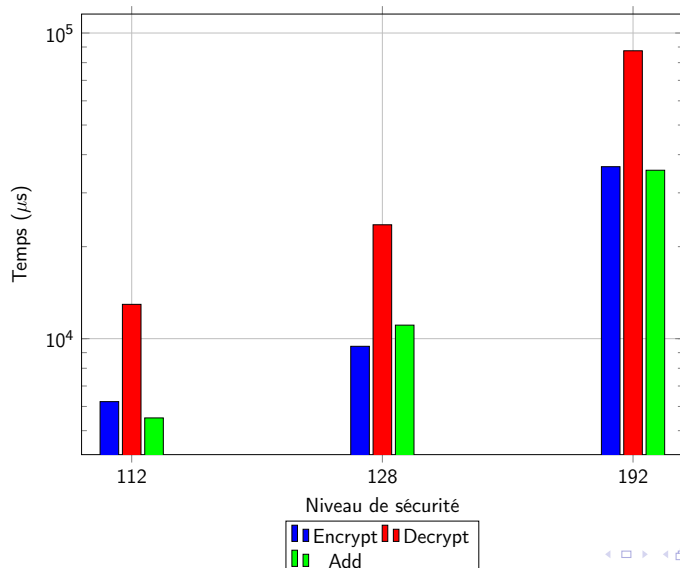
# Benchmark BICYCL vs Paillier (SecLevel 128)

|                | BICYCL | PHPE   | SHPE    | Gain  |
|----------------|--------|--------|---------|-------|
| CT (bits)      | 4 663  | 6 144  | 6 144   | +24%  |
| KeyGen (ms)    | 24.45  | 183.76 | 1353.04 | ×7–55 |
| Encrypt (ms)   | 9.44   | 84.75  | 163.63  | ×9–17 |
| Decrypt (ms)   | 23.58  | 22.36  | 3.74    | ×6    |
| Add ( $\mu$ s) | 11 073 | 70     | 70      | ×158  |

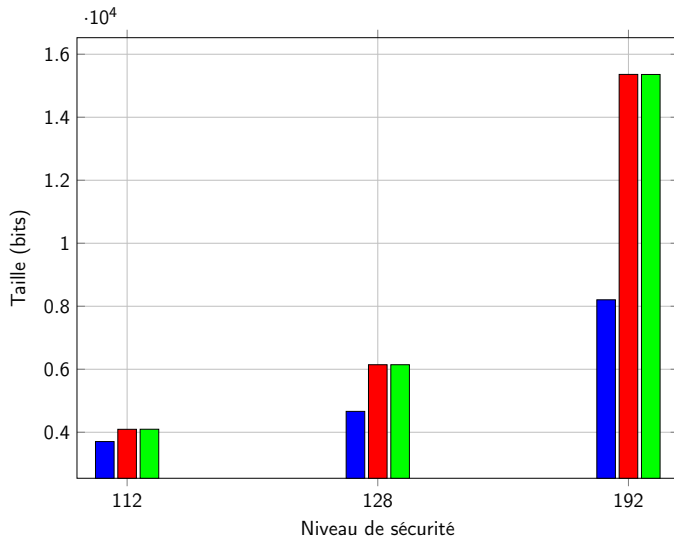
## Observations

- BICYCL : Chiffrés plus compacts, chiffrement plus rapide
- Paillier : Addition homomorphe plus rapide
- **Compromis favorable pour FPSU**

# Temps d'exécution BICYCL



# Taille des chiffrés



| Variante          | Setup (ms) | Décodage (ms) | Total (ms) |
|-------------------|------------|---------------|------------|
| FPSUServer (base) | 0.13       | 0.004         | 0.134      |
| FPSUServer_bicycl | 31.20      | 2.50          | 33.70      |
| <b>Facteur</b>    | ×240       | ×625          | ×251       |

## Analyse

- Surcoût : facteur  $\approx 250$
- Principalement dû au chiffrement (16 chiffrements  $\times 9.44$  ms)
- **Acceptable** pour des ensembles de taille modérée ( $n \leq 100$ )

# Résultats de décodage

## Test avec $n = 8$ , $m = 5$

- Indices 0 et 1 : Clés synchronisées → **Succès**
- Indice 2 : Clés désynchronisées → **Échec**
- Indices 3 et 4 : Clés aléatoires → **Échec**

## Sortie console

```
decode succeeded: The server already  
has the number 0  
decode succeeded: The server already  
has the number 1  
decode failed: The server does not  
have the number value 2  
decode failed: The server does not  
have the number value 3
```

## Interprétation

- Décodage OKVS correct
- Reconstruction homomorphe valide
- Détection des éléments hors ensemble

## Fonctionnalités

- Requêtes PIR sur les OKVS stockés
- Cache les indices demandés
- Deux serveurs indépendants

```
// Setup des serveurs PIR
PIRServerSetup(1, storage[0].size());
PIRServerSetup(2, storage[1].size());

// Requête client pour l'index 3
PIRClientQuery(3);
PIRServerAnswer(1);
PIRServerAnswer(2);
PIRClientExtract();

// D codage classique
paxos.decode<ValueType>(clientSets[0], decoded[0], storage[0]);
paxos.decode<ValueType>(clientSets[1], decoded[1], storage[1]);
```

# Comparaison BICYCL vs Paillier

Table – SecLevel 128,  $k = 256$  bits

| Critère         | BICYCL | Paillier | Vainqueur |
|-----------------|--------|----------|-----------|
| CT bits         | 4 663  | 6 144    | BICYCL    |
| KeyGen (ms)     | 24.45  | 183.76   | BICYCL    |
| Encrypt (ms)    | 9.44   | 84.75    | BICYCL    |
| Decrypt (ms)    | 23.58  | 22.36    | Paillier  |
| Add ( $\mu s$ ) | 11 073 | 70       | Paillier  |

## Conclusion

BICYCL est **compétitif** pour FPSU :

- Chiffrement plus rapide
- Clés plus compactes
- Meilleure intégration C++

## Complexité

- Chiffrement :  $\mathcal{O}(n)$
- Décodage :  $\mathcal{O}(m)$
- Reconstruction :  $\mathcal{O}(m)$
- **Linéaire en  $n$  et  $m$**

## Estimations ( $n$ éléments)

| $n$  | Temps estimé (ms) |
|------|-------------------|
| 8    | 33.7              |
| 100  | $\approx 420$     |
| 1000 | $\approx 4200$    |

## Scalabilité

- Linéaire en la taille des ensembles
- Parallélisation possible
- Adapté aux applications réelles

## Limites actuelles

- Taille fixe de `ValueType` (4096 octets)
- Surcharge de sérialisation
- Tests limités à  $n = 8$

## Améliorations possibles

- 1 Compression des QFI (plus compacte)
- 2 Batch encryption / decryption
- 3 Utilisation de  $k > 1$  pour plus de sécurité
- 4 Optimisation du  $\text{operator}^{\wedge}$  sur `ValueType`

## Travaux futurs

- **Extension** à de plus grands ensembles ( $n \geq 10^3$ )
- **Optimisation** de la sérialisation
- **Parallélisation** des opérations
- **Comparaison** avec d'autres schémas homomorphes
- **Formalisation** de la sécurité

## Applications

- **Médical** : Partage sécurisé de données patients
- **Réseau** : Détection collaborative de menaces
- **Finance** : KYC inter-bancaire
- **IA** : Apprentissage fédéré

## Contributions

- ① Implémentation de FPSUServer\_bicycl
- ② Intégration de BICYCL dans volePSI
- ③ Campagne de benchmarks BICYCL vs Paillier
- ④ Évaluation du protocole FPSU avec chiffrement homomorphe

## Résultats clés

- BICYCL : +24% de compacité,  $\times 9$  chiffrement
- Surcoût FPSU : facteur 250 (acceptable)
- **Compromis favorable** pour les applications modérées

**Merci pour votre attention !**

# Code source : sérialisation I

```
void ciphertext_to_valuetype(const BICYCL::CL_HSMqk::CipherText& ct, ValueType& out)
{
    std::vector<unsigned char> payload;
    std::vector<unsigned char> c1_bytes = encode_qfi(ct.c1());
    std::vector<unsigned char> c2_bytes = encode_qfi(ct.c2());
    payload.insert(payload.end(), c1_bytes.begin(), c1_bytes.end());
    payload.insert(payload.end(), c2_bytes.begin(), c2_bytes.end());

    std::fill(out.begin(), out.end(), ZeroBlock);
    std::uint32_t payload_len = static_cast<std::uint32_t>(payload.size());
    const size_t header_size = sizeof(payload_len);
    if (header_size + payload.size() > kMaxValueBytes) {
        throw std::runtime_error("Serialized ciphertext is too large");
    }

    std::memcpy(out.data(), &payload_len, header_size);
    std::memcpy(reinterpret_cast<unsigned char*>(out.data()) + header_size,
                payload.data(), payload.size());
}
```

# Code source : reconstruction homomorphe I

```
for (u64 i = 0; i < m; ++i)
{
    bool found = false;
    try {
        BICYCL::CL_HSMqk::CipherText c1 =
            valuetype_to_ciphertext(crypto_system, decoded1[i]);
        BICYCL::CL_HSMqk::CipherText c2 =
            valuetype_to_ciphertext(crypto_system, decoded2[i]);

        BICYCL::CL_HSMqk::CipherText c_sum =
            crypto_system.add_ciphertexts(public_key, c1, c2, randgen);

        BICYCL::CL_HSMqk::ClearText decrypted_sum =
            crypto_system.decrypt(secret_key, c_sum);

        BICYCL::Mpz zero(0u1);
        found = (decrypted_sum == zero);
    } catch (const std::exception&) {
        found = false;
    }
}
```

## Code source : reconstruction homomorphe II

```
if (found)
    std::cout << "decode succeeded: The server already has the number " << i <<
        "\n";
else
    std::cout << "decode failed: The server does not have the number value " <<
        i << "\n";
}
```

## Paramètres

- $q$  : module (256 bits pour SecLevel 128)
- $k$  : dimension du sous-groupe
- $\Delta_K$  : discriminant du corps quadratique

## Opérations

- **KeyGen** : Génère  $(sk, pk)$
- **Encrypt** :  $\mathcal{E}(m) \in \text{Cl}(\Delta_K)^2$
- **Decrypt** :  $\mathcal{D}(c) = m$
- **Add** :  $\mathcal{E}(m_1) \oplus \mathcal{E}(m_2) = \mathcal{E}(m_1 + m_2)$